

Absolutely. When teaching **React.js security**, it helps to separate **React-specific practices** from general web/API security.

Important React.js Security Practices

#	Practice	What to do
1	Prevent XSS	Avoid <code>dangerouslySetInnerHTML</code> ; sanitize HTML if you absolutely need to render it.
2	Never store sensitive data in <code>localStorage</code>	Avoid storing passwords, API secrets, or long-lived sensitive tokens there because JavaScript can access them.
3	Use HttpOnly cookies for sensitive auth tokens	HttpOnly cookies cannot be directly read by JavaScript, reducing token theft through XSS.
4	Validate input	Validate user input on the frontend for UX, but always validate again on the backend .
5	Don't trust frontend authorization	Hiding an admin button with React does not prevent someone from calling the API directly. Authorization must happen on the server.
6	Protect API requests	Use HTTPS and proper authentication/authorization for API calls.
7	Don't expose secrets in React	Anything bundled into frontend JavaScript can potentially be seen by users. Never put database passwords or private API keys in React code.
8	Keep dependencies updated	Regularly check packages with <code>npm audit</code> and update vulnerable dependencies.
9	Use a Content Security Policy (CSP)	CSP can significantly reduce the impact of XSS and injected scripts.
10	Protect against CSRF	Particularly important when authentication uses cookies. Use appropriate SameSite settings and/or CSRF tokens.
11	Avoid rendering untrusted URLs	Don't blindly use user-controlled values in <code><a href></code> , <code><iframe src></code> , redirects, etc.

#	Practice	What to do
12	Don't leak sensitive information in errors	Avoid displaying stack traces, tokens, database errors, or internal API information to users.
13	Secure authentication flows	Use secure password handling, MFA where appropriate, session expiration, logout/invalidation, and rate limiting on the backend.
14	Use security headers	Configure headers such as CSP, X-Content-Type-Options, Referrer-Policy, and appropriate frame protections.
15	Separate frontend and backend responsibilities	React handles UI/input validation; the backend remains the source of truth for authentication, authorization, validation, and business rules.

★ The 5 I would emphasize to students

If you're teaching a React class, I'd focus on these first:

1. XSS

JavaScript

```
// ❌ Dangerous
<div dangerouslySetInnerHTML={{ __html: userInput }} />
```

Prefer:

JavaScript

```
// ✅ React escapes this automatically
<div>{userInput}</div>
```

2. Don't put secrets in React

JavaScript

```
// ❌ Never
const DB_PASSWORD = "mySuperSecretPassword";
```

Even environment variables used by a frontend build are **not automatically secret**.

JavaScript

```
// ⚠️ This can still be exposed in the browser
const apiKey = import.meta.env.VITE_API_KEY;
```

If the browser needs it, assume the user can inspect it.

3. Don't rely on React for authorization

JavaScript

```
// ❌ This is NOT security
{user.role === "admin" && <DeleteButton />}
```

A malicious user can still send:

http

```
DELETE /api/users/123
```

Your backend must verify:

```
Is the user authenticated?
    ↓
Does the user have permission?
    ↓
Allow / Reject request
```

4. Be careful with tokens

A common approach is:

JavaScript

```
localStorage.setItem("token", token);
```

It's convenient, but an XSS vulnerability can potentially allow malicious JavaScript to read that token.

For many web applications, a more secure pattern is:

Browser

↓

HttpOnly + Secure + SameSite cookie

↓

Backend

The exact authentication architecture depends on the application.

5. Keep dependencies secure

Run:

Bash

```
npm audit
```

and keep your dependencies updated.

Also avoid installing packages blindly—every dependency becomes part of your application's attack surface.

One important rule

React security is not just about React.

A secure React application usually looks like:

```
React
|
Input validation
Safe rendering
|
HTTPS
|
Backend API
|
Authentication
Authorization
Input validation
Rate limiting
|
Database
```

Frontend security improves safety, but the backend must always be trusted—not the React client.